Parameter-Efficient Supervised Fine-Tuning for Multi-Horizon Prognostics in Electric Commercial Vehicle Chassis Systems

The Prognostics Imperative in Electric Commercial Vehicles

The transition from internal combustion engine (ICE) vehicles to distributed-drive electric vehicles (DDEVs) has fundamentally altered the physical, thermodynamic, and computational landscape of commercial fleet management. In electric truck chassis systems, the over-actuated nature of independent wheel hub motors presents unique opportunities for precise torque vectoring, but it simultaneously introduces highly complex, coupled dynamics.[1] The immense unsprung mass of battery storage systems, combined with aggressive steering and regenerative braking maneuvers, leaves EV trucks highly susceptible to latent structural fatigue, suspension degradation, and catastrophic subsystem failures.[1]

Historically, fleet maintenance has relied upon reactive diagnostics, identifying a fault only after a Diagnostic Trouble Code (DTC) is triggered via On-Board Diagnostics (OBD) systems or when a driver perceives a physical abnormality.[3] However, mechanical degradation in critical safety components often precedes electronic detection, leading to secondary damage, unscheduled downtime, and severe safety risks before an alert is ever registered.[3] The contemporary engineering paradigm has consequently shifted toward prognostics: the continuous, probabilistic estimation of a component's Remaining Useful Life (RUL).[4]

Prognostics aims to transcend simple anomaly detection by forecasting the exact trajectory of degradation over multiple temporal horizons. For modern logistics, it is insufficient to know that a system is currently failing; fleet operators must mathematically project the state of the chassis 1 day, 1 week, 1 month, and 1 year into the future.[7] For an EV truck, calculating the RUL of steering assemblies, friction braking systems, and hydraulic brake fluid reservoirs requires the synthesis of high-fidelity telemetry, including friction work approximated from longitudinal braking forces, inertial measurement unit (IMU) data, and state-of-charge (SoC) parameters.[4]

Achieving such predictive capability traditionally necessitates training massive deep neural networks from scratch on immense, proprietary datasets, an endeavor that is computationally prohibitive, environmentally taxing, and prone to overfitting on narrow fleet distributions. To resolve this bottleneck, the field is rapidly adopting foundation models for time-series forecasting. By leveraging pre-trained sequential architectures and applying

Parameter-Efficient Fine-Tuning (PEFT) methodologies such as Low-Rank Adaptation (LoRA), engineers can adapt massive temporal models to specific EV chassis fault domains using minimal computational resources.[12] This report details the theoretical, mathematical, and programmatic architecture required to implement an exhaustive multi-horizon RUL forecasting pipeline using Keras, transforming raw chassis telemetry into actionable, probabilistic maintenance horizons.

Physics and Failure Modes of EV Truck Chassis Systems

To construct a robust predictive model, the algorithm must be grounded in the physical realities of the mechanical systems it monitors. Electric vehicle chassis architectures experience stress profiles that differ significantly from legacy ICE vehicles. The integration of heavy battery packs alters the center of gravity, while regenerative braking systems modify the thermal cycling of traditional hydraulic friction brakes.[1]

The first critical failure domain involves brake faults and brake fluid leakage. In traditional vehicles, hydraulic brakes dissipate kinetic energy entirely through friction, generating substantial heat. In EV trucks, braking is a hybrid process where the electric motor reverses its operation to act as a generator, capturing kinetic energy and routing it back to the battery (regenerative braking), while the hydraulic friction brakes supply the remaining required stopping force.[10] While this reduces overall brake pad wear, it introduces complex failure modes. If the regenerative system fails or is restricted due to a fully charged battery (high SoC), the friction brakes must suddenly absorb massive, unanticipated thermal loads, leading to accelerated pad glazing or wear.[10] Furthermore, the hydraulic fluid lines are subjected to immense pressure spikes during aggressive combined maneuvers. A microscopic fissure in the brake line may manifest as a slow, imperceptible drop in hydraulic line pressure over several weeks.[11] If left unaddressed, this slow brake fluid leakage eventually leads to vapor lock or complete loss of hydraulic pressure, resulting in catastrophic brake failure. A prognostic model must identify the subtle, long-term decay in pressure retention (the 1-month horizon) and predict the exact date of critical failure (the 1-day or 1-week horizon) based on these telemetry signatures.

The second domain encompasses steering faults and suspension degradation. Electric trucks frequently utilize active steering and active lateral stabilizer bars composed of electronic servo valve hydraulic actuators to manage roll stability and mitigate the risk of rollover accidents.[2] These systems are heavily taxed by the massive weight of the vehicle. Steering faults often originate as micro-vibrations or increased resistance in the steering column, detectable via high-frequency anomalies in the lateral acceleration and yaw rate sensors.[2] Similarly, suspension dampers and wheel hubs degrade invisibly. A broken damper with partial oil leakage or a noisy, rusted wheel hub may not trigger an immediate OBD error, but it drastically alters the hysteresis curve of the suspension system.[3] The vibrations propagate through the

chassis, eventually compromising the steering geometry and leading to sudden directional instability during high-speed obstacle avoidance maneuvers.[2] By applying spectral analysis and deep learning to the Fast Fourier Transform (FFT) of wheel speed signals, these mechanical decay patterns can be identified months before they cause a macro-level steering failure.[3]

Telemetry Ecosystems and Prognostic Datasets

The efficacy of any predictive maintenance framework is inextricably linked to the quality, granularity, and multivariate nature of the underlying temporal data. Constructing a multi-horizon forecasting model for EV truck faults requires data that captures not only instantaneous dynamic events but also long-term, cyclical wear patterns.[6]

To establish robust baseline models, researchers and engineers utilize large-scale repositories that mirror the operational realities of heavy-duty vehicles. The SCANIA Component X dataset represents a foundational benchmark for predictive maintenance.[17] Extracted from a fleet of 33,641 SCANIA trucks, this dataset provides a rich integration of operational telemetry, repair records, and static vehicle specifications.[6] The temporal data includes diverse features presented as histograms and numerical counters, gathered directly from on-board sensors that monitor the real-time condition of the truck's engine and chassis components.[6] The inclusion of time-to-event information allows for the precise mathematical formulation of RUL targets, making it ideal for supervised fine-tuning.[6]

Furthermore, the EVIoT-PredictiveMaint dataset offers telemetry explicitly tailored to the electric vehicle domain.[11] This dataset isolates the specific stressors that affect EV chassis systems, tracking over thirty distinct features categorized by subsystem.[11]

Telemetry Category	Critical Sensor Signals	Prognostic Relevance to Chassis Health
Brake System Dynamics	Brake pad wear thickness, hydraulic pressure, brake fluid levels.	Directly maps to mechanical degradation and the risk of catastrophic stopping failure, identifying microscopic fluid leaks before a pressure collapse.[11]
Steering and Suspension	Tire pressure, lateral acceleration, yaw rate, damper displacement.	Identifies broken dampers and noisy wheel hubs prior to catastrophic structural

		failure, predicting steering column misalignment.[2]
Electric Powertrain	Motor temperature, State of Charge (SoC), regenerative braking torque.	Fluctuations in regenerative torque distribution heavily influence brake pad thermal cycling and wear rates, altering the RUL of the friction brakes.[10]
Environmental & Usage	Ambient temperature, cumulative distance traveled, payload weight.	Modulates the baseline rate of degradation; heavier chassis architectures operating in extreme cold degrade faster under identical operational loads.[11]

In practical deployments, data is logged via Controller Area Network (CAN) bus systems using standards such as the FIBEX database format, often capturing high-frequency inputs like wheel speed signals at 5 to 20 Hz.[3] To utilize this data in a Keras-based neural network, it must be down-sampled, normalized, and windowed. Keras provides utilities such as tf.keras.utils.timeseries_dataset_from_array to convert continuous time-series data into discrete input-label pairs using sliding window generators.[7] For a 1-year predictive horizon, historical data is aggregated into daily or weekly summaries to prevent the sequence length from exceeding the maximum context window of the transformer model, while short-term predictions for the 1-day horizon retain minute-by-minute granularity to capture sudden dynamic shifts in fluid pressure or steering resistance.[22]

Deep Learning Architectures for Time-Series Prognostics in Keras

The Keras ecosystem supports a multitude of architectures capable of processing multivariate time series, ranging from classic Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks to modern Transformer-based foundation models.[7] While LSTMs natively handle sequential data and possess gating mechanisms to retain long-term memory, they struggle with the vanishing gradient problem over the extended lookback windows required for 1-year RUL predictions, and they lack the innate ability to prioritize varying input modalities efficiently.[23] Consequently, attention-based architectures have become the standard for robust, multi-horizon RUL prediction.[25]

The Temporal Fusion Transformer (TFT) is uniquely engineered for multi-horizon forecasting, blending high-performance prediction with inherent interpretability.[25] Unlike natural language

transformers that treat all tokens equally, the TFT is explicitly designed to handle heterogeneous time-series features.[16] The TFT categorizes input data into distinct types, which is critical for EV truck chassis modeling. Time-varying known inputs encompass data predictable in the future, such as scheduled maintenance dates or seasonal temperature shifts. Time-varying unknown inputs consist of data generated strictly up to the present moment, such as real-time brake fluid pressure or lateral acceleration. Static categorical features include identifiers that do not change, such as the specific truck model or chassis configuration, while static real features represent unchanging numerical metrics, such as the initial unladen weight of the vehicle.[16]

To process this disparate data, the TFT utilizes specialized Variable Selection Networks (VSNs) that assign dynamic weights to each feature, filtering out irrelevant noise. For example, the VSN learns to ignore ambient temperature if it currently exhibits no correlation with a specific steering fault, focusing instead on yaw rate anomalies.[26] The architecture then applies standard LSTM layers for local temporal processing, followed by interpretable multi-head self-attention layers to identify long-range dependencies, such as correlating a harsh braking event that occurred three months prior with current brake pad failure.[25]

Recent advancements mirror the evolution of Large Language Models (LLMs), leading to the creation of Time-Series Foundation Models. These are massive, pre-trained networks capable of zero-shot forecasting, meaning they can predict degradation in an EV truck chassis without having been explicitly trained on EV data, relying instead on generalized temporal patterns learned from vast corpora of open-source time-series data.[27]

Models like Amazon's Chronos treat time-series forecasting as a language modeling task.[29] Raw continuous telemetry, such as brake wear measurements or fluid leak rates, is transformed into a sequence of discrete tokens via scaling and quantization.[29] These tokens are fed into an encoder-only architecture and trained using a cross-entropy loss function.[27] Chronos models, available in sizes up to hundreds of millions of parameters, natively support multi-step-ahead forecasting and cross-learning across related covariates.[27] Similarly, architectures like PatchTST segment long time series into localized patches, feeding these subsets into a transformer to learn local semantic meanings while preserving channel independence, which is vital for monitoring independent wheel hub motors.[30]

The introduction of KerasHub provides an integrated framework to load these pre-trained architectures seamlessly, allowing developers to instantiate complex backbones with a single line of code and map them to their specific chassis telemetry arrays.[32] Keras provides access to pre-trained models via the keras_hub.models API, enabling rapid deployment of foundational logic.[32]

The Mathematics of Multi-Horizon Forecasting

Standard time-series models typically output a single, one-step-ahead prediction.[7] For basic diagnostic purposes, identifying a current fault state is sufficient. However, estimating RUL requires anticipating system degradation across multiple, distinct future horizons. A fleet manager must know if a brake fluid leak will cause a critical failure in 1 day, requiring immediate vehicle grounding; in 1 week, requiring route alteration; in 1 month, allowing for scheduled depot maintenance; or in 1 year, factoring into long-term capital depreciation.[8]

Forecasting paradigms dictate how these horizons are generated. Autoregressive or iterative forecasting involves the model predicting the state at time step $t+1$, feeding that prediction back into the input sequence, and recursively predicting $t+2$.[7] While conceptually simple, this method suffers from severe compounding error over long horizons; a slight overestimation in brake pad wear at day 7 will exponentially distort the predicted state at month 6.[7] Conversely, single-shot direct forecasting maps a historical input sequence directly to a vector of future outputs representing the entire horizon.[7] This mitigates compounding errors but scales poorly when attempting to predict granular data points over a massive span.

Advanced architectures employ Direct Multi-Horizon Forecasting. By utilizing sequence-to-sequence networks augmented with context vectors, the architecture separates horizon-specific contexts from horizon-agnostic contexts.[34] The neural network branch responsible for global feature extraction summarizes the encoder output plus all future inputs into two contexts: a series of horizon-specific contexts for each of the future points, and a horizon-agnostic context that captures common, underlying information.[34] This bifurcation allows the model to accurately predict both tomorrow's sudden degradation and next year's long-term wear simultaneously.[34]

Probabilistic RUL Estimation via Quantile Regression

When estimating the Remaining Useful Life of a critical safety system like an EV truck braking assembly, a single point estimate (e.g., predicting exact failure in 42 days) is dangerously inadequate. Due to the stochastic nature of road conditions, payload variances, driver behavior, and environmental stressors, prognostic models must output probabilistic distributions to quantify uncertainty.[36] Fleet operations necessitate knowing the earliest possible failure date (the pessimistic bound), the most likely failure date (the median expectation), and the maximum optimal lifespan (the optimistic bound).[37]

Traditional neural networks optimize for Mean Squared Error (MSE), which predicts the conditional mean of the target variable. This optimization heavily penalizes large outliers but fundamentally assumes that the variance of the residuals is constant. This assumption fails dramatically in mechanical wear scenarios, where degradation rates accelerate non-linearly and variance explodes near the end of a component's life.[16] To generate highly calibrated prediction intervals, the Keras architecture must substitute MSE with a Pinball Loss or Quantile

Loss function.[36] Quantile regression estimates the conditional median and asymmetric percentiles.[16]

For a given prediction $\hat{y}$, actual observation y, and target quantile $q \in (0, 1)$, the quantile loss L_q is defined mathematically as:

$$L_q(y, \hat{y}) = \max(q(y - \hat{y}), (1 - q)(\hat{y} - y))$$

This piecewise function applies asymmetric penalties depending on the specific quantile being trained. If a Keras model is tasked with predicting the 90th percentile ($q = 0.9$) of brake pad lifespan, the loss function heavily penalizes underestimations by a massive factor of 0.9, while only lightly penalizing overestimations by a factor of 0.1.[16] This forces the network to learn the upper bound of the distribution. Conversely, setting $q = 0.1$ trains the network to learn the lower bound, severely penalizing overestimations.

To train a multi-horizon forecasting model for probabilistic output, the final loss function is aggregated across a predetermined set of quantiles, such as $q \in \{0.10, 0.50, 0.90\}$.[16]

$$L_{total} = \sum_{q \in Q} \sum_{t=1}^{K} L_q(y_t, \hat{y}_t^{(q)})$$

Where K represents the various multi-horizon steps (1 day, 1 week, 1 month, 1 year). This Constrained Weighted Quantile Loss ensures that the model learns a comprehensive probability density function for the RUL of the chassis, guiding equipment maintenance decisions and effectively avoiding the catastrophic economic losses associated with unexpected on-road breakdowns.[38]

Target Quantile	Statistical Interpretation	Maintenance Strategy Implication for EV Chassis
P10 (10th Percentile)	Pessimistic / Early Failure Bound	Defines the absolute latest date to schedule maintenance to avoid critical safety violations. If a brake fluid leak trajectory crosses the minimum pressure threshold at the

		P10 1-week horizon, immediate routing to a depot is triggered.[37]
P50 (50th Percentile)	Median / Expected RUL	Serves as the primary operational baseline for supply chain inventory ordering. Parts such as replacement steering column actuators are ordered based on the P50 1-month horizon.[16]
P90 (90th Percentile)	Optimistic / Maximum Lifespan	Indicates the theoretical limit of component integrity under perfect operational conditions; utilized primarily for long-term fleet depreciation modeling and 1-year financial forecasting.[37]

Parameter-Efficient Fine-Tuning (PEFT) via Low-Rank Adaptation (LoRA)

While pre-trained foundation models and generalized time-series Transformers offer exceptional baseline capabilities, they must be fine-tuned to understand the highly specific nuances of a commercial EV chassis.[13] A generalized model will not inherently understand that an EV truck experiencing a 5% drop in regenerative braking torque will simultaneously experience accelerated hydraulic brake pad wear, nor will it understand the specific telemetry signature of a noisy wheel hub.[3]

Historically, supervised fine-tuning involved unfreezing the entire neural network and updating millions or billions of parameters via backpropagation. This methodology requires immense GPU clusters, astronomical memory overhead, and runs the severe risk of catastrophic forgetting, wherein the model loses its generalized predictive abilities while over-optimizing for a small dataset.[14] To circumvent these limitations, the paradigm has decisively shifted toward Parameter-Efficient Fine-Tuning (PEFT) techniques.[41]

Low-Rank Adaptation (LoRA) stands as the most prominent and effective PEFT methodology.[12] LoRA operates on the underlying principle that the updates to a model's weights during fine-tuning have a low intrinsic rank, meaning the necessary mathematical changes can be

represented in a significantly lower-dimensional space without sacrificing prognostic accuracy.[14]

In a standard dense layer or attention projection matrix within a Keras model, the pre-trained weight matrix $W_0 \in \mathbb{R}^{n \times d}$ is updated during training by a gradient matrix ΔW. LoRA alters this architecture by freezing the original weight matrix W_0 entirely. It then injects two smaller, trainable dense matrices into the specific layer: Matrix $A \in \mathbb{R}^{n \times r}$ and Matrix $B \in \mathbb{R}^{r \times d}$, where r is the designated rank and is significantly smaller than the original dimensions ($r \ll n, d$).[14]

The forward pass equation is thus modified from the standard $output = W_0 x + b_0$ to the LoRA-adapted equation:

$$output = W_0 x + b_0 + BAx$$ [14]

Matrix A is typically initialized with random Gaussian variance to break symmetry, while Matrix B is strictly initialized with zeros. This strategic initialization ensures that at the very start of the fine-tuning process, the product BAx equates to exactly zero, meaning the model behaves identically to its pre-trained state prior to any weight updates.[42] An additional scaling factor, α/r, is applied to the output of the BAx matrix multiplication to control the magnitude of the updates.[42]

The impact on computational efficiency for an EV chassis prognostics model is profound. If an attention projection layer in a time-series Transformer has an input dimension of 768 and an output dimension of 768, standard fine-tuning requires tracking and updating 589,824 parameters for that single layer.[14] By applying LoRA with a rank of $r = 4$, the trainable matrices consist solely of $A(768 \times 4)$ and $B(4 \times 768)$, totaling only 6,144 parameters.[14] This represents a near 99% reduction in trainable weights for that specific structural component.

By drastically reducing the optimizer state, which usually stores massive moment vectors for every trainable parameter, and minimizing the backward pass memory footprint, LoRA enables the fine-tuning of massive predictive maintenance models on consumer-grade hardware or single-instance cloud GPUs.[40] Furthermore, because the original foundation weights remain entirely frozen, multiple different LoRA adapters can be trained for specific failure modes. A

fleet operator can maintain one master time-series foundation model in Keras and dynamically swap in a specialized steering-fault LoRA adapter or a brake-fault LoRA adapter during inference, depending on which specific chassis sub-system is currently being analyzed by the diagnostic scheduler.[12]

Architecting the Prognostics Pipeline in Keras

Implementing this sophisticated pipeline involves orchestrating Keras data handlers, instantiating a backbone time-series model from KerasHub or custom layer definitions, injecting custom LoRA layers, and compiling the model with multi-quantile outputs.[14]

The initialization phase begins with data structuring. The raw telemetry from the EVIoT or SCANIA datasets must be transformed into continuous sliding windows. Using the Keras tf.keras.utils.timeseries_dataset_from_array utility, the high-dimensional data is sliced into input matrices spanning the historical lookback period, mapped directly to label matrices spanning the multi-step future horizons.[7] To predict up to 1 year without exhausting sequence length limits, the model pipeline utilizes hierarchical subsampling: the 1-day predictions rely on highly granular minute-by-minute data to catch sudden fluid leaks, while the 1-year prediction relies on down-sampled weekly averages to track long-term suspension damper degradation.[3]

If utilizing an architecture like the Temporal Fusion Transformer built from scratch, the model requires a complex multi-input Keras functional API setup.[43] Continuous variables, such as motor temperature or hydraulic pressure, pass through dense linear embeddings, while categorical variables, such as the specific truck identification number, pass through keras.layers.Embedding layers.[43] Alternatively, utilizing a KerasHub pre-trained backbone vastly simplifies this step. A general-purpose transformer backbone can be loaded directly using keras_hub.models, with its pre-trained weights automatically frozen by the API to preserve the foundational temporal logic.[32]

Because native LoRA support for highly customized time-series regression models often requires architectural specificities not found in standard NLP wrappers, a custom LoraLayer must be defined by subclassing the base keras.layers.Layer.[42] The initialization of the custom layer extracts the input and output dimensions of the target pre-trained layer, typically a dense projection layer within a transformer block. It subsequently initializes the two low-rank dense layers, A and B.[42]

This custom class wrapper is then systematically injected into the dense feed-forward networks and query/value projection matrices of the attention blocks by iterating over the model.layers attribute.[42] By overriding these specific layers and ensuring that layer.trainable = False for all non-LoRA components, the model is fully prepared for parameter-efficient fine-tuning without disturbing the pre-calculated internal representations.[42]

The final dense layer of the Keras model must be adapted to output a matrix of dimensions

$K \times Q$, where K represents the four temporal horizons (1 day, 1 week, 1 month, 1 year) and Q represents the number of target quantiles (P10, P50, P90).[34] The model is compiled using a custom loss function that implements the mathematically defined Constrained Weighted Quantile Loss across all outputs.[38] To optimize training time and memory bandwidth further during the model.fit() stage, Keras mixed precision, specifically bfloat16, is activated.[14] This prevents the GPU memory from bottlenecking during the computation of the extended sequential time-series sequences, a common failure point when processing months of high-frequency telemetry.[14]

During the execution of supervised fine-tuning, the backpropagation algorithm ignores the millions of parameters locked inside the pre-trained foundation model and focuses solely on calculating gradients for the minuscule LoRA matrices.[40] The model converges rapidly on the specific fault characteristics of the EV truck datasets, learning the intricate relationship between regenerative braking anomalies and hydraulic failure.[10] To prevent overfitting to the specific trucks in the training set, callbacks such as keras.callbacks.EarlyStopping are implemented, monitoring the validation quantile loss and halting training when generalization performance plateaus.[16]

Strategic Implications and Lifecycle Management

The culmination of this Keras-based fine-tuning pipeline yields a highly optimized set of adapter weights specific to the prognostics of EV truck chassis faults. Post-training evaluation involves rigorously analyzing the calibration of the prediction intervals: determining if the actual mechanical failures observed in the holdout test set fall within the P10 to P90 bounds approximately 80% of the time.[45] Models that exhibit overly wide confidence intervals are penalized for a lack of precision, as they provide no actionable timeline, while models with overly narrow intervals are penalized for a lack of reliability, as they risk missing critical early failures.

For deployment within the automotive industry, the computational efficiency of LoRA presents a unique architectural advantage. The parameter updates generated by Matrix A and Matrix B can be mathematically merged back into the frozen W_0 matrix using simple addition prior to deployment.[42]

$$W_{merged} = W_0 + (B \times A) \times scale$$

This merging process ensures that the deployed model incurs zero inference latency overhead compared to the original, unmodified foundation model, as the architecture returns to its original structural dimensions.[42] The finalized Keras model can then be exported via TensorFlow

Lite or optimized using XLA (Accelerated Linear Algebra) compilers for deployment directly onto the vehicle's Edge computing node, or integrated into centralized cloud-based fleet management dashboards via standard REST APIs.[46]

By continuously streaming CAN bus data through the lightweight inference model, fleet operators gain a dynamic, continuously updating timeline of their EV chassis health.[5] As the system detects microscopic shifts in torque response, brake fluid pressure, or steering yaw rates, the predicted RUL distributions adjust in real-time. A 1-year estimated lifespan for a suspension damper might violently contract to a 1-week pessimistic horizon if the model detects a sudden, sustained spike in hysteresis correlating with an impending structural failure.[3]

This transition from diagnostic reaction to multi-horizon prognostic anticipation fundamentally secures the operational stability, financial predictability, and life-critical safety of next-generation electric commercial fleets. By leveraging pre-trained time-series foundation models, implementing rigorous parameter-efficient fine-tuning via LoRA, and deploying quantile regression for probabilistic bounds, engineers can construct prognostic systems capable of mastering the complex physical dynamics inherent to distributed-drive electric vehicle chassis architectures.

Works cited

1. Chassis Control Methodologies for Steering-Braking Maneuvers in Distributed-Drive Electric Vehicles - ResearchGate, accessed March 8, 2026, https://www.researchgate.net/publication/400211036_Chassis_Control_Methodologies_for_Steering-Braking_Maneuvers_in_Distributed-Drive_Electric_Vehicles
2. Braking failure anti-rollover control and hardware-in-the-loop ..., accessed March 8, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11608256/
3. Chassis predictive maintenance and service ... - Diva-Portal.org, accessed March 8, 2026, http://www.diva-portal.org/smash/get/diva2:1379746/FULLTEXT01.pdf
4. Estimation of brake pad wear and remaining useful life from fused sensor system, statistical data processing, and passenger car longitudinal dynamics, accessed March 8, 2026, https://orbit.dtu.dk/en/publications/estimation-of-brake-pad-wear-and-remaining-useful-life-from-fused
5. Predictive Maintenance for Truck Fleets | AI in Transport Logistics by KEBA DIGITAL, accessed March 8, 2026, https://www.keba.com/en/keba-digital/products-services/logistics/predictive-maintenance
6. SCANIA Component X dataset: a real-world multivariate time series dataset for predictive maintenance - PMC, accessed March 8, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11933314/
7. Time series forecasting | TensorFlow Core, accessed March 8, 2026, https://www.tensorflow.org/tutorials/structured_data/time_series
8. Multi-label machine learning for power forecasting of a grid-connected

photovoltaic solar plant over multiple time horizons - PMC, accessed March 8, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12457606/
9. Multi-horizon time series prediction with neural networks - Decide Soluciones, accessed March 8, 2026, https://decidesoluciones.es/en/multi-horizon-time-series-prediction-with-neural-networks/
10. Simulation Research on Torque Vectoring, Braking, Chassis Control Strategies for Electric & Hybrid Vehicles using System Simulation - Technical Paper - SAE Mobilus, accessed March 8, 2026, https://saemobilus.sae.org/papers/simulation-research-torque-vectoring-braking-chassis-control-strategies-electric-hybrid-vehicles-using-system-simulation-2026-26-0423
11. EVIoT-PredictiveMaint Dataset - Kaggle, accessed March 8, 2026, https://www.kaggle.com/datasets/datasetengineer/eviot-predictivemaint-dataset
12. LoRA methods - Hugging Face, accessed March 8, 2026, https://huggingface.co/docs/peft/task_guides/lora_based_methods
13. Fine-tune Gemma in Keras using LoRA | Google AI for Developers, accessed March 8, 2026, https://ai.google.dev/gemma/docs/core/lora_tuning
14. Parameter-efficient fine-tuning of Gemma with LoRA and QLoRA - Keras, accessed March 8, 2026, https://keras.io/examples/keras_recipes/parameter_efficient_finetuning_of_gemma_with_lora_and_qlora/
15. Driver and vehicle behaviour to power train failures in electric vehicles – experimental results of field and simulator studie - DiVA, accessed March 8, 2026, https://www.diva-portal.org/smash/get/diva2:1367953/FULLTEXT01.pdf
16. Temporal Fusion Transformer: Time Series Forecasting with Deep ..., accessed March 8, 2026, https://towardsdatascience.com/temporal-fusion-transformer-time-series-forecasting-with-deep-learning-complete-tutorial-d32c1e51cd91/
17. SCANIA Component X Dataset: A Real-World Multivariate Time Series Dataset for Predictive Maintenance - Researchdata.se, accessed March 8, 2026, https://researchdata.se/en/catalogue/dataset/2024-34/1
18. (PDF) An Empirical Study on Predictive Maintenance for Component X in Heavy-Duty Scania Trucks - ResearchGate, accessed March 8, 2026, https://www.researchgate.net/publication/393420496_An_Empirical_Study_on_Predictive_Maintenance_for_Component_X_in_Heavy-Duty_Scania_Trucks
19. SCANIA Component X Dataset: A Real-World Multivariate Time Series Dataset for Predictive Maintenance - arXiv, accessed March 8, 2026, https://arxiv.org/html/2401.15199v1
20. Predictive Maintenance of vehicles in the Automotive Industry - DEV Community, accessed March 8, 2026, https://dev.to/ludwig023/predictive-maintenance-of-vehicles-in-the-automotive-industry-46d5
21. Predicting the Market: A Hands-On Guide to Time-Series Forecasting with Python - Medium, accessed March 8, 2026,

https://medium.com/@writeronepagecode/predicting-the-market-a-hands-on-guide-to-time-series-forecasting-with-python-4f4b1bec6359
22. How to Predict the future values of time horizon with Keras? - Data Science Stack Exchange, accessed March 8, 2026, https://datascience.stackexchange.com/questions/30762/how-to-predict-the-future-values-of-time-horizon-with-keras
23. Programming LSTM with Keras and TensorFlow (10.2) - YouTube, accessed March 8, 2026, https://www.youtube.com/watch?v=wY0dyFgNCgY
24. Multivariate Time Series Forecasting with LSTMs in Keras - MachineLearningMastery.com, accessed March 8, 2026, https://machinelearningmastery.com/multivariate-time-series-forecasting-lstms-keras/
25. mattsherar/Temporal_Fusion_Transform: Pytorch Implementation of Google's TFT - GitHub, accessed March 8, 2026, https://github.com/mattsherar/Temporal_Fusion_Transform
26. Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting, accessed March 8, 2026, https://www.personal.soton.ac.uk/cz1y20/Reading_Group/mlts-2023/week3/TFT.pdf
27. amazon/chronos-2 - Hugging Face, accessed March 8, 2026, https://huggingface.co/amazon/chronos-2
28. Chronos: The Latest Time Series Forecasting Foundation Model by Amazon, accessed March 8, 2026, https://www.datasciencewithmarco.com/blog/chronos-the-latest-time-series-forecasting-foundation-model-by-amazon
29. Chronos: Learning the language of time series - Amazon Science, accessed March 8, 2026, https://www.amazon.science/code-and-datasets/chronos-learning-the-language-of-time-series
30. yuqinie98/PatchTST: An offical implementation of PatchTST: "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers." (ICLR 2023) https://arxiv.org/abs/2211.14730 · GitHub - GitHub, accessed March 8, 2026, https://github.com/yuqinie98/PatchTST
31. Recent Advances in Transformers for Time-Series Forecasting : r/datascience - Reddit, accessed March 8, 2026, https://www.reddit.com/r/datascience/comments/1egw3ij/recent_advances_in_transformers_for_timeseries/
32. keras-team/keras-hub: Pretrained model hub for Keras 3. - GitHub, accessed March 8, 2026, https://github.com/keras-team/keras-hub
33. Introducing Keras Hub: Your one-stop shop for pretrained models - Google Developers Blog, accessed March 8, 2026, https://developers.googleblog.com/en/introducing-keras-hub-for-pretrained-models/
34. A Multi-Horizon Quantile Recurrent Forecaster - Amazon Science, accessed March 8, 2026,

https://assets.amazon.science/e2/53/153ad5064cf2b95ba4d73d659527/a-multi-horizon-quantile-recurrent-forecaster.pdf
35. [1711.11053] A Multi-Horizon Quantile Recurrent Forecaster - arXiv, accessed March 8, 2026, https://arxiv.org/abs/1711.11053
36. Quantile regression, from linear models to trees to deep learning | by Max Ghenis - Medium, accessed March 8, 2026, https://medium.com/data-science/quantile-regression-from-linear-models-to-trees-to-deep-learning-af3738b527c3
37. From Scratch to Deep Quantile Forecasting | by Jinhang Jiang | TDS Archive - Medium, accessed March 8, 2026, https://medium.com/data-science/from-scratch-to-deep-quantile-forecasting-366d84b7dd22
38. Additive Ensemble Neural Network with Constrained Weighted Quantile Loss for Probabilistic Electric-Load Forecasting - PMC, accessed March 8, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC8123084/
39. Predicting Remaining Useful Life of Rolling Bearings Based on Reliable Degradation Indicator and Temporal Convolution Network with the Quantile Regression - MDPI, accessed March 8, 2026, https://www.mdpi.com/2076-3417/11/11/4773
40. Efficient Fine-tuning with PEFT and LoRA | Niklas Heidloff, accessed March 8, 2026, https://heidloff.net/article/efficient-fine-tuning-lora/
41. PEFT - Hugging Face, accessed March 8, 2026, https://huggingface.co/docs/peft/index
42. Parameter-efficient fine-tuning of GPT-2 with LoRA - Keras, accessed March 8, 2026, https://keras.io/examples/nlp/parameter_efficient_finetuning_of_gpt2_with_lora/
43. Zenome84/tft_keras: A version of the Temporal Fusion ... - GitHub, accessed March 8, 2026, https://github.com/Zenome84/tft_keras
44. Single custom Loss function for Multiple Output Regression not working · Issue #1216 · rstudio/keras3 - GitHub, accessed March 8, 2026, https://github.com/rstudio/keras/issues/1216
45. Fast and accurate zero-shot forecasting with Chronos-Bolt and AutoGluon - Amazon AWS, accessed March 8, 2026, https://aws.amazon.com/blogs/machine-learning/fast-and-accurate-zero-shot-forecasting-with-chronos-bolt-and-autogluon/
46. (PDF) AI-Driven Predictive Maintenance Framework for Next- Generation Electric Mobility Systems - ResearchGate, accessed March 8, 2026, https://www.researchgate.net/publication/396403286_AI-Driven_Predictive_Maintenance_Framework_for_Next-_Generation_Electric_Mobility_Systems